SwiftDrop Parcel Delivery and Courier Management System

Final Submission Report

Group Minveli_2.0 — Faculty of Information Technology, University of Moratuwa

15 November 2026

Index number	Name
234202J	Suthankan B.
234006L	Abhinaya J.
235115D	Vibhushan G.
234210G	Thuvarahan T.
234065N	Gajanthan S.
234214X	Pushmitha V.
234197U	Sinthuha N.
234077D	Harithkeshan R.
234163M	Praveen APE

Table 1. Group roster

Contents

Abstract Introduction Requirement Coverage Final Design Summary Implementation The System in Use Testing and Evaluation Demonstration Plan Conclusion References Appendix A – Individual Contributions Appendix B – Running the System Appendix C – Main API Endpoints

List of Figures

Figure 1. Logical container architecture Figure 2. Kafka reliability and SSE fan-out Figure 3. Google login via Keycloak Figure 4. Courier status update and webhook Figure 5. Kubernetes deployment

List of Tables

Table 1. Group roster Table 2. Functional requirements Table 3. System requirements Table 4. Implementation, testing and bonus requirements Table 5. Integration patterns Table 6. Demonstration steps Table 7. Main API endpoints

1 Abstract

SwiftDrop is a parcel delivery and courier management platform for an island-wide courier company that previously ran on phone calls and spreadsheets. Customers request fixed-price quotes, book parcels and follow them live; online merchants register, are approved by an administrator and create orders from their own systems through scoped API keys, receiving signed webhooks when parcel status changes; couriers work through their daily assignments and record proof or failure reasons; dispatchers assign, reassign and monitor couriers; and administrators manage users, merchants, zones, pricing and reports.

Our report of 1 November 2026 presented the analysis and design (use cases, architecture, activity and sequence diagrams, data models and justifications) together with an initial code base containing working logins. This final report describes the complete system delivered for the final submission. The platform consists of four business services, an API gateway, a read-only GraphQL query service and an AI assistant, each built with Spring Boot 4.1 on Java 21, a React 19 single-page application, Keycloak with Google sign-in, PostgreSQL per service, Apache Kafka with a transactional outbox, server-sent events for live tracking, and MinIO for private proof photos. It runs with one command through Docker Compose and is also packaged as a Helm chart for Kubernetes.

The report maps the implementation to the assignment brief and records the automated and runtime evidence available for the submission. Kubernetes packaging and the AI delivery assistant are included as bonus deliverables.

2 Introduction

2.1 Background

Last-mile delivery involves many parties who need timely and trustworthy information about the same parcel. SwiftDrop's customers had no visibility of where their parcels were, and its partner online stores wanted to create delivery orders automatically from their own e-commerce platforms. The assignment asked us to design and build a modern enterprise application for this scenario: secure, containerised, tested and integrated with external systems.

2.2 Aim and objectives

The aim was to build a working parcel delivery and courier management platform that meets the assignment brief. The objectives were to:

1. support every actor in the brief: customer, merchant, courier, dispatcher and administrator;
2. integrate with external systems: Google sign-in, merchant systems through APIs and webhooks, and e-mail;
3. provide real-time tracking and event-driven notifications;
4. protect personal data and enforce role-based and ownership-based access on every request;
5. containerise the system and make it runnable with one command, with a Kubernetes path;
6. demonstrate quality through automated tests, measurable coverage and documented evidence;
7. attempt both bonus items: Kubernetes deployment and an AI delivery assistant.

2.3 From the November 1 milestone to the final system

The November 1 submission contained the full design package and an initial code base in which every role could sign in (local accounts and Google), with the gateway and services validating tokens. For the final milestone the team integrated the business functions of every service, the integration flows, the front end for all roles, the AI assistant, the Kubernetes packaging, and two rounds of system-wide auditing with their fixes. The service boundaries were retained. The final design and release work covered the Same Day service tier, dispatcher reassignment and unassignment, customer self-service of contact details, and an optional shared-database mode on Supabase for team development.

2.4 Structure of this report

Chapter 2 maps every requirement of the brief to the delivered system. Chapter 3 summarises the final design; the full design is in the accompanying *Design Documentation* PDF. Chapter 4 describes the implementation of each part of the system. Chapter 5 shows the system in use with screenshots for every role. Chapter 6 presents testing and evaluation results. Chapter 7 sets out the demonstration plan. Chapter 8 concludes with achievements, problems, limitations and further work. The appendices give each member's contribution, instructions for running the system and the main API endpoints.

3 Requirement Coverage

The tables below map each requirement to where it is delivered.

3.1 Functional requirements

Brief requirement	Delivered
Secure signup and login, including Google (OAuth 2.0)	Keycloak with Authorization Code + PKCE; Google identity provider; e-mail verification; password policy
Request a delivery quote	Pricing service: fixed 15-minute quotes, Standard or Same Day, size bands, chargeable weight
Real-time tracking	Server-sent events through the gateway, with REST refresh on reconnect
Delivery history and receipts	Customer history page; PDF receipt after delivery
Mobile-friendly public tracking page	Public tracking by number; shows status and coarse zones only, never addresses
Merchant registration with admin approval	Merchant registration in Accounts; admin review with approve, reject and suspend

Brief requirement	Delivered
Orders via a secured API (API keys or OAuth 2.0 client credentials)	Scoped, hashed, expiring API keys (`X-API-Key`) validated at the gateway; idempotent order API
Webhooks on status change	Signed HTTPS webhooks (HMAC-SHA256) with up to five retries and backoff; delivery logs; secret rotation
Merchant dashboard	Orders dashboard with summary cards, filters, CSV export, bulk CSV import and order details
Courier: today's assignments	Courier space lists today's assignments
Courier: update stages with notes and proof	Picked Up → In Transit → Out for Delivery → Delivered; notes; photo proof required for delivery
Courier: failed delivery with reason	Stage-specific failure reasons (e.g. recipient unavailable, parcel damaged)
Dispatcher: view unassigned and assign	Unassigned queue with zone filter; eligibility-checked assignment; reassign and unassign before pickup
Monitor courier workloads	Active assignments and capacity per courier
Manage users, merchants, zones and pricing	Admin users and staff, merchant review, zone editor and versioned rate cards
Reports: deliveries per day, success/failure rate	Admin and dispatcher reports with per-day breakdown, success and failure rates, zone filter

Table 2. Functional requirements

3.2 System requirements

Brief requirement	Delivered
Real-time updates (WebSockets, SSE or polling)	SSE for parcel tracking
Role-based access control	Five realm roles enforced at the gateway and again in every service, plus ownership and tenant checks
JWT authentication, OAuth 2.0 for social login or merchant API	Keycloak JWTs; Google via OAuth 2.0 / OpenID Connect
Asynchronous integration via a message broker	Kafka topic `parcel.lifecycle.v1` with a transactional outbox, consumer groups and a dead-letter topic
Notification service consuming status events	Communications service sends e-mail (and mock SMS) from Kafka events
Containerised deployment	Dockerfiles for every service; Docker Compose for the whole stack; Helm chart
Secure handling of personal data	Bean Validation, password hashing in Keycloak, no personal data in logs, errors or events

Table 3. System requirements

3.3 Implementation, testing and bonus requirements

Brief requirement	Delivered
Responsive React UI	React 19 + TypeScript SPA, responsive layouts and a mobile tracking page
Spring Boot backend	Spring Boot 4.1 on Java 21 for all services
Versioned, validated APIs with centralised error handling	`/api/v1` REST and GraphQL; Bean Validation; RFC 9457 Problem Details in every service
Broker, notification consumer, webhooks with retry	See system requirements
Code quality and configuration	Environment variables only; Compose refuses to start without required secrets
Unit and integration tests incl. an event-flow test	JUnit and Testcontainers suites per service; outbox/Kafka and webhook pipeline tests
Authorisation tests (e.g. courier cannot access another's deliveries)	Gateway, controller and gRPC authorisation suites; live checks
Measurable coverage with results	JaCoCo per Java module, Vitest coverage for the front end (Chapter 5)
Bonus: Kubernetes with ConfigMaps, Secrets, Services, Ingress, Helm	Helm chart with all of these plus NetworkPolicies and PodDisruptionBudgets
Bonus: AI delivery assistant, MCP encouraged	Gemini-based assistant calling backend tools; same tools served over MCP

Table 4. Implementation, testing and bonus requirements

4 Final Design Summary

The complete design, with all diagrams, is in the accompanying *SwiftDrop Design Documentation* (PDF). This chapter summarises it.

4.1 Architecture

SwiftDrop is built as microservices split by business capability. Four business services (Accounts & Partners, Pricing & Zones, Delivery Operations and Communications) each own their PostgreSQL database. A Spring Cloud Gateway is the single entry point; a read-only GraphQL Query API composes dashboards from the services over gRPC; and an AI assistant service answers questions using the same APIs. Keycloak handles identity, Kafka carries status events, and MinIO stores proof photos privately.

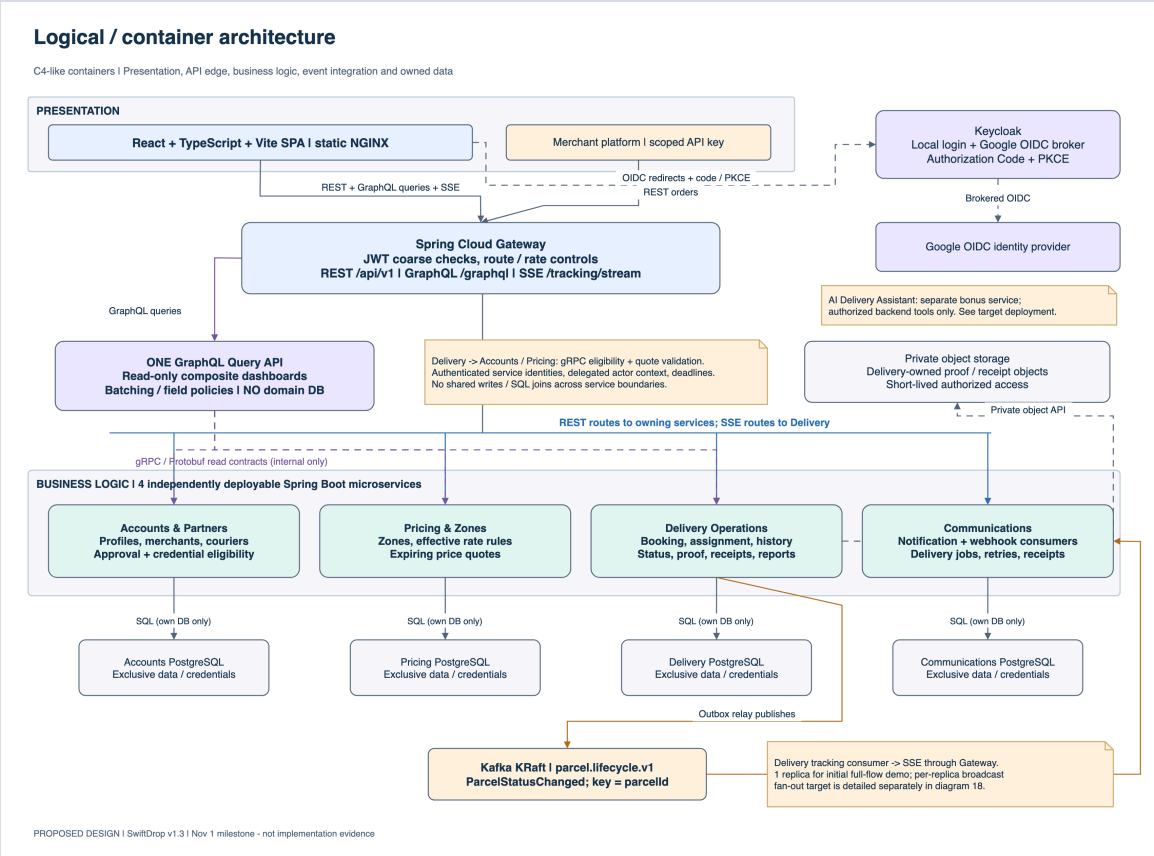


Figure 1. Logical container architecture

The choice of microservices gave each pair of developers a clear area to own, isolated slow external integrations (merchant webhooks, e-mail) from booking and tracking, and kept personal data inside the service that owns it. We accepted the extra operational work of running more containers in return.

4.2 Integration patterns

Pattern	Use	Reason
Request/reply (REST)	Commands from the browser and merchant systems	Immediate, validated answers; universal for clients
Request/reply (gRPC)	Quote validation, courier eligibility, dashboard reads	Typed contracts and deadlines between services
API gateway and query composition	Spring Cloud Gateway; GraphQL Query API	One secured entry point; one round trip per dashboard
Publish/subscribe	Kafka status events to notifications, webhooks and tracking	Several independent reactions without slowing the courier
Transactional outbox, idempotent receiver, dead letter	Delivery outbox; idempotency keys; DLT and quarantine	No lost or duplicated effects under failure
Webhook callback with retry	Signed merchant callbacks	Push to merchants; signatures prevent forgery
Server push	SSE for live tracking	Simple one-way updates

Table 5. Integration patterns

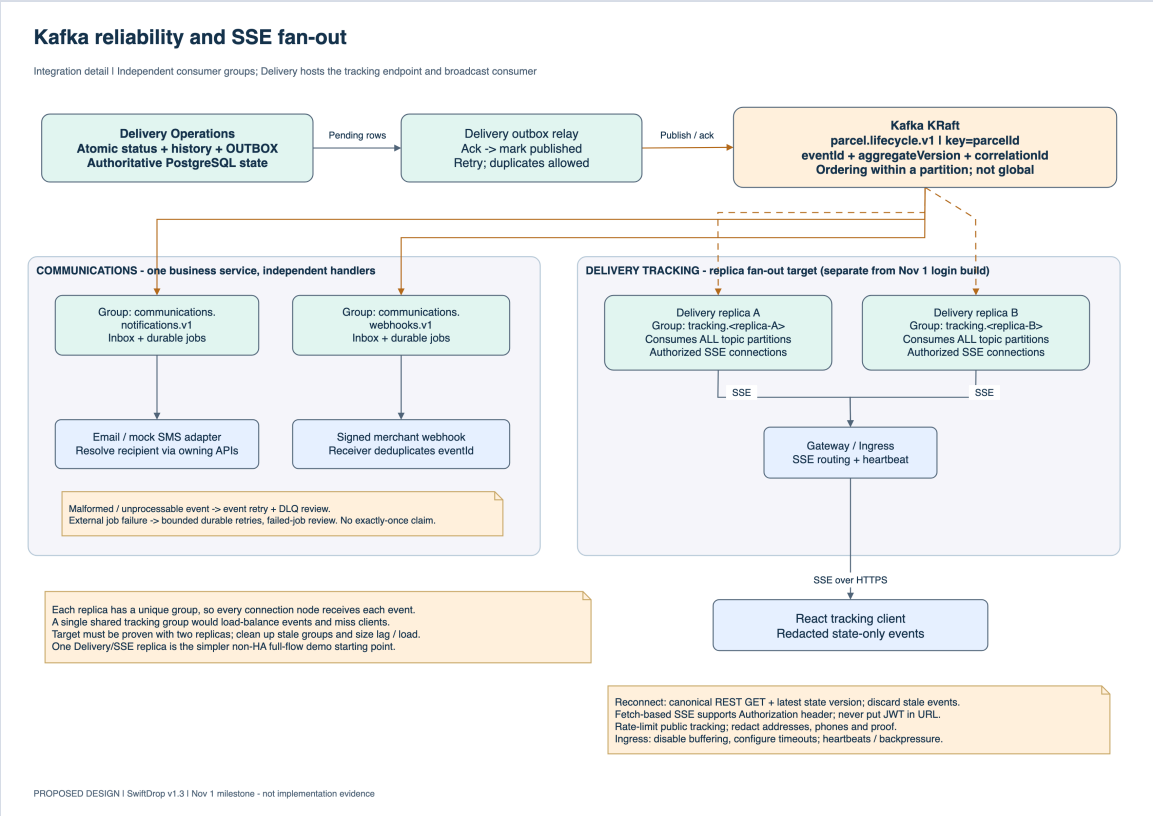


Figure 2. Kafka reliability and SSE fan-out

4.3 Key flows

The design documentation contains the activity and sequence diagrams for all key flows. The OAuth 2.0 login flow and the courier status update that triggers notifications are reproduced here.

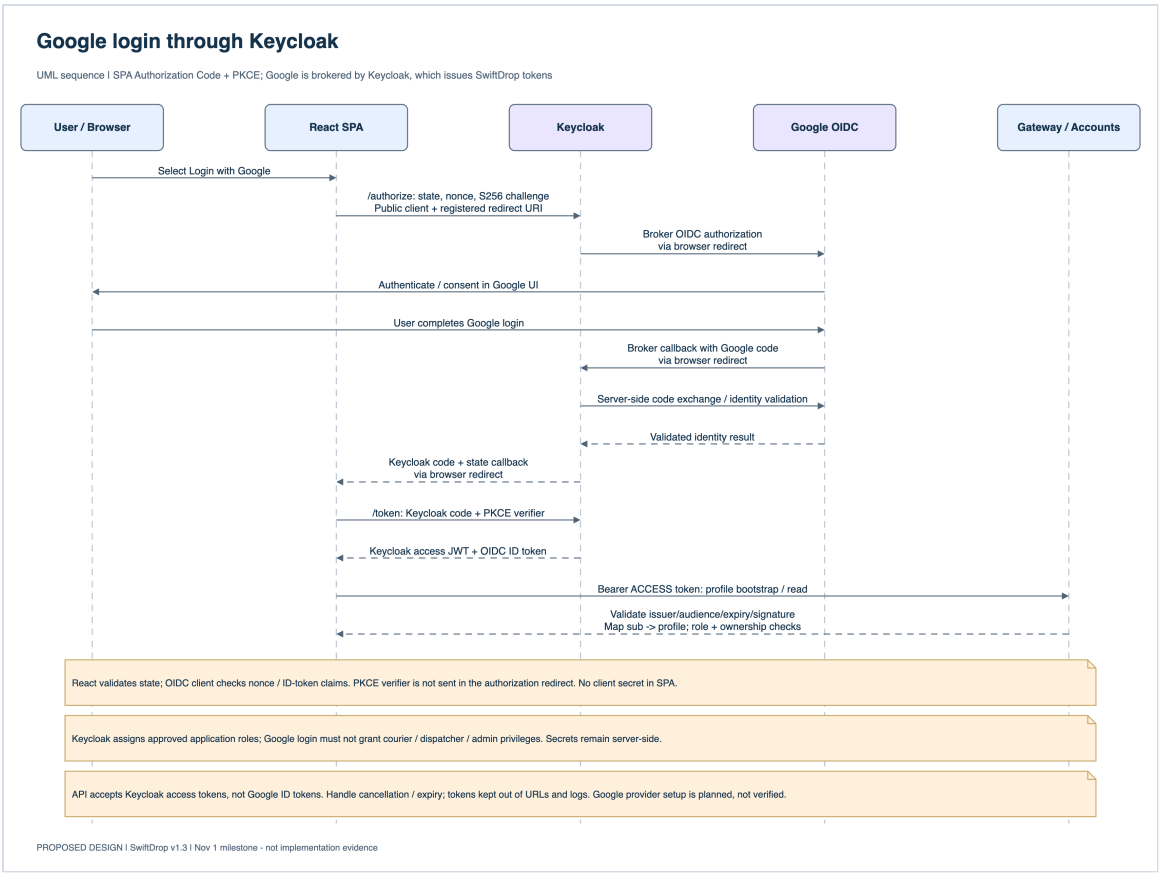


Figure 3. Google login via Keycloak

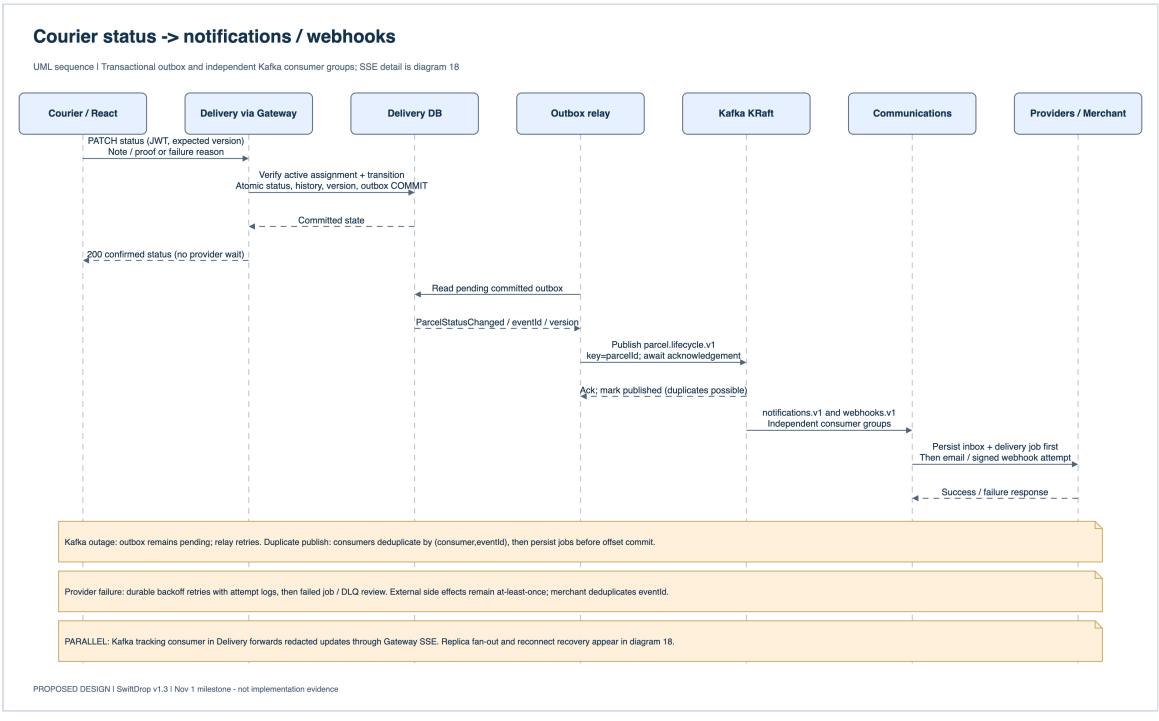


Figure 4. Courier status update and webhook

4.4 Changes to the design since 1 November

- **Same Day service tier.** Quotes can be Standard or Same Day with a surcharge. Pricing passes the tier to Delivery through an additive field in the gRPC contract, and it is shown to customers, merchants and couriers.
- **Reassign and unassign.** Dispatchers can change an assignment while the parcel has not been picked up; every change is audited.
- **Customer self-service.** Users can update their own name and phone number.
- **AI assistant service.** Added as a separate service behind the gateway (Chapter 4).
- **Shared development databases.** An optional Supabase mode lets the team share databases; a fresh clone still runs entirely locally.

5 Implementation

5.1 Repository and way of working

The project is a monorepo: `frontend/`, `services/` (one folder per Spring Boot service), `ai-assistant/`, `shared/` (security adapter and Protobuf contracts), `contracts/` (REST, GraphQL, gRPC and event schemas), `platform/` (Compose, identity, Kubernetes and infrastructure), `tests/` (contract, end-to-end, security and policy suites) and `docs/`. Work was done in feature branches merged through pull requests into `test` and promoted to `main`. A GitHub Actions pipeline builds every module, runs the tests, checks contracts and scans images and dependencies.

5.2 Authentication and edge security

Keycloak hosts the `swiftdrop` realm with five roles, a SwiftDrop login theme, e-mail verification and the Google identity provider. The React application uses `keycloak-js` with PKCE and keeps tokens in memory only. The gateway validates every token, enforces route-level roles, validates merchant API keys with Accounts over gRPC, applies rate limits to public endpoints, strips untrusted headers and refuses tokens of users an administrator has disabled. Every service validates the token again with a shared security library and checks ownership of the data requested.

5.3 Accounts & Partners

Accounts keeps user and staff profiles keyed by the Keycloak subject, merchant registrations with an audited approval workflow, and merchant API keys stored as hashes with scopes and expiry. It provides courier eligibility, merchant tenant resolution and permitted contact details to other services over gRPC, and lets administrators provision staff, disable users and reconcile roles with Keycloak.

5.4 Pricing & Zones

Pricing manages delivery zones and versioned rate cards (draft, active, superseded, archived). Quotes are calculated by a `PricingStrategy` (tiered by size band and chargeable weight, the higher of physical and volumetric weight), include an optional Same Day surcharge, are fixed for 15 minutes and are validated by Delivery over gRPC before booking.

5.5 Delivery Operations

Delivery is the source of truth for parcels. Booking validates the quote and stores the parcel, its first history entry, an idempotency record and an outbox event in one transaction. Status changes follow the lifecycle state machine with optimistic locking; delivery requires a photo proof stored privately in MinIO. Delivery manages assignments (one active per parcel, with audit), produces PDF receipts, serves public and authenticated tracking, streams live updates over SSE, and relays outbox events to Kafka.

5.6 Communications

Communications consumes status events from Kafka. It sends e-mail notifications through SMTP (Mailpit locally) with recorded attempts and replay, and delivers merchant webhooks: HTTPS only, checked against server-side request forgery, signed with HMAC-SHA256, retried up to five times with exponential backoff and marked exhausted afterwards. Merchants can read delivery logs and rotate secrets; duplicate endpoints are refused. Poison events are quarantined for administrator replay.

5.7 GraphQL Query API and front end

The GraphQL Query API provides read-only dashboards for customers, merchants, dispatchers and administrators, and delivery statistics for reports, by calling the services over gRPC with the caller's delegated identity and strict complexity limits. The React front end provides a space for each role, a public tracking page, live tracking, the reports page and the help assistant, with validation, accessible forms and plain-language messages.

5.8 AI delivery assistant

The assistant is a separate Spring Boot service behind the gateway. It sends the conversation to Google Gemini with tool definitions (track a parcel, get a quote, list zones and, for signed-in users, list the user's own parcels), executes the calls the model requests against SwiftDrop's APIs with the caller's own token, and returns the answer. It is read-only, strips addresses before results reach the model, and exposes the same tools through the Model Context Protocol (JSON-RPC) for other AI clients. Without a Gemini key the front end falls back to keyword answers.

5.9 Deployment and continuous integration

Every service has a Dockerfile. `platform/compose/start.sh` starts the whole system: PostgreSQL servers, Kafka, MinIO, Mailpit, Keycloak with the imported realm, and all services, with generated per-installation secrets and health checks. All required secrets come from environment variables, and Compose refuses to start without them. For the Kubernetes bonus, a Helm chart deploys the system with ConfigMaps, Secrets, Services, an Ingress, NetworkPolicies and PodDisruptionBudgets, tested on a local k3d cluster, with a gated Azure K3s path in Terraform. The Azure path is preparation for a possible deployment.

Local verification uses the Compose PostgreSQL containers. The opt-in shared development configuration introduced in PR #325 documents three Supabase projects and five independent PostgreSQL databases with dedicated service logins. Accounts and Keycloak each have their own project. Pricing, Delivery and Communications use separate databases in the business project. The team reported successful shared development reads and writes. The submission checks use the local database mode. See shared development setup.

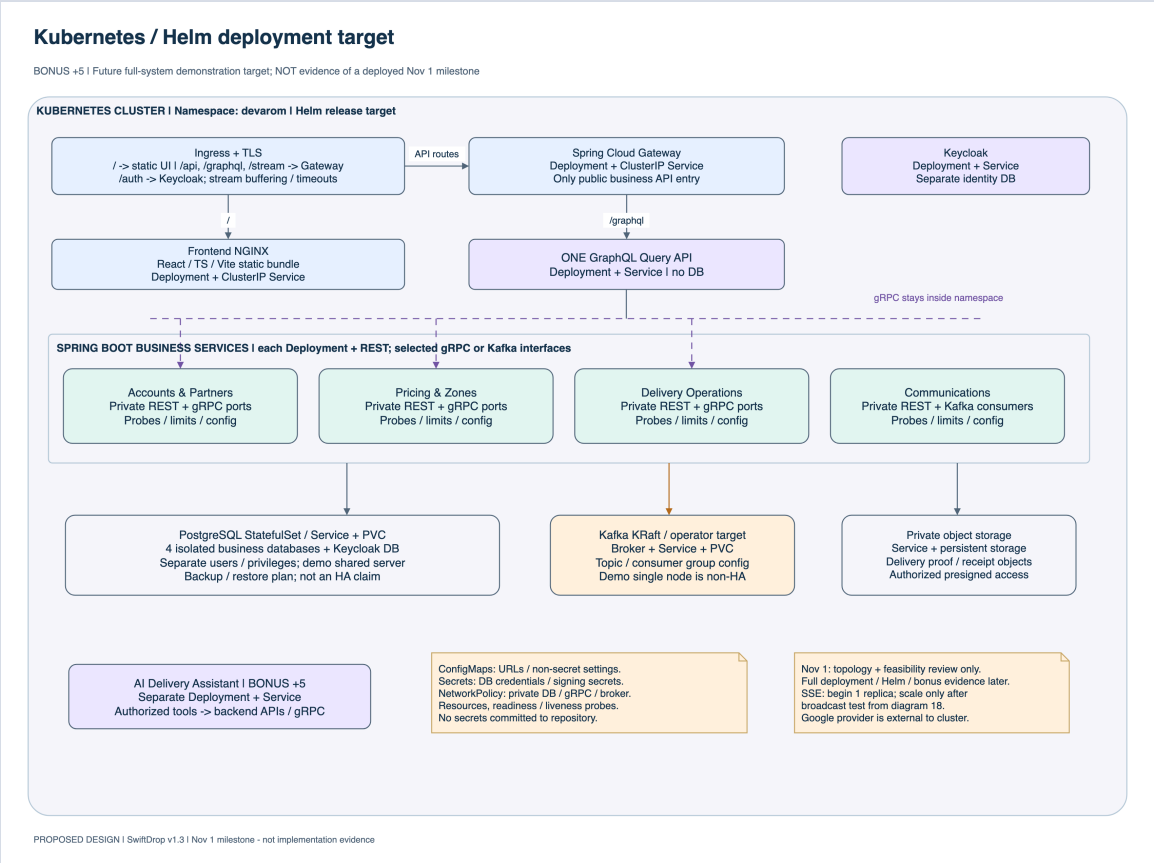


Figure 5. Kubernetes deployment

6 The System in Use

The isolated local PostgreSQL stack was prepared for a five-role walkthrough. Keycloak did not become available during startup of the release images on the Apple Silicon test host, so demo-account sign-in and authenticated screen capture remain pending. The demonstration plan lists the intended walkthrough.

7 Testing and Evaluation

The release source for this report is GitHub test commit 9768726. The 13 November GitHub Actions run completed successfully, including frontend quality and build, all Java service jobs, policy and contract checks, security scans, and the required CI gate. See the separate test report for the run scope and local verification results.

The baseline and repository test report describe earlier evaluation. This report bases final-release claims on the linked commit and current evidence. Google sign-in, external email delivery, live provider responses and deployment to Azure require separate environment-specific evidence.

8 Demonstration Plan

The demonstration uses the local Compose stack and the demo accounts created by `platform/identity/seed-demo-accounts.py` (one per role, with an approved merchant).

Brief demo item	Steps
Customer login incl. Google, delivery creation, real-time tracking	Sign in with Google; get a Same Day quote; book; open tracking while the courier updates the parcel
Merchant order via the API and receiving a webhook	Create an API key; create an order with `curl` and `X-API-Key`; show the webhook delivery log
Dispatcher assigning a parcel; courier updating status	Assign the parcel; as the courier mark it picked up, in transit and out for delivery, upload proof and deliver
Notification triggered by a status change	Show the e-mail in Mailpit and the live update on the customer's screen
Walkthrough of architecture, diagrams, integration and security	Use the Design Documentation PDF
Bonus	Ask the help assistant where a parcel is and how much a delivery costs; show the Helm chart

Table 6. Demonstration steps

9 Conclusion

9.1 Achievements

SwiftDrop brings together five role-based portals and a service architecture with independently owned databases. The release passed the repository CI gate. The demonstration plan describes the steps for evaluating the end-to-end flows.

9.2 Problems encountered

- **Keeping nine members' work compatible.** Shared Protobuf and event contracts, a required CI gate and contract compatibility tests kept branches aligned.
- **Resource limits on development laptops.** Running eight Java services with Kafka and Keycloak on a 4 GB Docker host caused out-of-memory failures; we capped heap sizes, limited test parallelism in image builds and added an optional shared-database mode.
- **External provider limits.** A Gemini model was retired during the project and the free API tier is rate-limited; the assistant now uses a current model, reports when the provider is busy and falls back to keyword answers when the provider is unavailable.
- **Tooling interference.** IDE background builds writing into Maven output folders broke some local builds; clean builds and isolated build directories avoided this.

9.3 Limitations

- Merchants authenticate machine calls with API keys; OAuth 2.0 client credentials (the brief's alternative) are not offered.
- SMS notifications are mocked.
- The Kubernetes deployment is verified on a local cluster; cloud deployment is prepared but gated by our zero-cost hosting decision.
- The AI assistant depends on an external provider with usage limits. Its live responses are outside the evidence collected for this report.

9.4 Further work

- OAuth 2.0 client credentials for merchants.
- A real SMS provider.
- Courier route suggestions.
- Store plugins (for example for WooCommerce or Shopify) so that non-technical merchants can connect without writing code.
- A production deployment with managed databases and monitoring.

10 References

1. Apache Software Foundation (2025), *Apache Kafka Documentation*, https://kafka.apache.org/documentation/
2. Evans, E. (2003), *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley.
3. Fielding, R. T. (2000), *Architectural Styles and the Design of Network-based Software Architectures*, PhD dissertation, University of California, Irvine.
4. Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley.
5. GraphQL Foundation (2021), *GraphQL Specification, October 2021 Edition*, https://spec.graphql.org/October2021/
6. gRPC Authors (2025), *gRPC Documentation*, https://grpc.io/docs/
7. Hardt, D. (2012), *The OAuth 2.0 Authorization Framework*, RFC 6749, IETF.
8. Hohpe, G. and Woolf, B. (2003), *Enterprise Integration Patterns*, Addison-Wesley.
9. Jones, M., Bradley, J. and Sakimura, N. (2015), *JSON Web Token (JWT)*, RFC 7519, IETF.
10. Keycloak Project (2025), *Keycloak Server Administration Guide*, https://www.keycloak.org/documentation
11. Model Context Protocol (2025), *Model Context Protocol Specification*, https://modelcontextprotocol.io/
12. Newman, S. (2021), *Building Microservices*, 2nd edition, O'Reilly Media.
13. Nottingham, M., Wilde, E. and Dalal, S. (2023), *Problem Details for HTTP APIs*, RFC 9457, IETF.
14. OWASP Foundation (2023), *OWASP API Security Top 10 – 2023*, https://owasp.org/API-Security/
15. Richardson, C. (2018), *Microservices Patterns*, Manning Publications.
16. Sakimura, N. et al. (2014), *OpenID Connect Core 1.0*, OpenID Foundation.
17. Sakimura, N., Bradley, J. and Agarwal, N. (2015), *Proof Key for Code Exchange by OAuth Public Clients*, RFC 7636, IETF.
18. Standard Webhooks (2024), *Standard Webhooks Specification*, https://www.standardwebhooks.com/
19. WHATWG (2025), *HTML Living Standard, Server-sent events*, https://html.spec.whatwg.org/multipage/server-sent-events.html

11 Appendix A – Individual Contributions

Each member's description is based on the work recorded in the repository (pull requests and commits) and on the members' own accounts in the November 1 report, updated for the final phase. The contribution sheet is available separately (`docs/submission/CONTRIBUTION_SHEET.md`).

Suthankan B. (234202J) — Platform, Security and Cross-Module Integration Lead. Owned all authentication: the Keycloak realm, login theme, e-mail verification, optional TOTP and Google broker; the React OIDC client with PKCE; gateway JWT validation and the shared service security library. Built the platform: Docker images, the Compose runtime, the CI pipeline with security scanning and contract checks, and the Kubernetes stage with Helm, monitoring, backups and recovery drills. In the final phase led the integration fixes and security and privacy hardening across services, added the Same Day tier, admin user controls with gateway enforcement, dispatcher reassignment, the AI delivery assistant with MCP tools, the two system audits and their fixes, the design documentation and this submission package.

Abhinaya J. (234006L) — Courier Module, Front End. Built the guarded courier landing page, the mobile-friendly assigned-deliveries list, the authorised parcel details and the stage and notes actions with conflict recovery; implemented the courier dashboard GraphQL resolver and the policy-gated proof retention cleanup; added accessibility, session-recovery and failure-recovery tests.

Vibhushan G. (235115D) — Courier Module, Backend and Independent Security QA. Independent security tester for the login milestone. In Delivery Operations implemented guarded transitions with append-only history, guarded assignment, the transactional outbox with a bounded Kafka relay, proof metadata and the full lifecycle; implemented authenticated SSE streaming with multi-replica broadcast, webhook egress rotation and replay, and the multi-actor authorisation regression suite.

Thuvarahan T. (234210G) — Merchant Module, Backend. Implemented the merchant approval state and tenant authorisation provider, the tenant-bound webhook subscription API, audited merchant approval and suspension, and tenant-bound merchant booking with idempotency; fixed quote snapshot mapping, secret-scanner findings and vulnerable dependencies, and added correlation IDs and bounded metrics.

Gajanthan S. (234065N) — Customer Module, Front End and Service Foundations. Helped freeze the Protobuf contracts, established the Delivery Operations foundation and the Accounts gRPC reads; implemented the customer quote and booking screens, the dashboard summary cards, authenticated tracking with SSE recovery and public tracking; integrated business routes into the gateway.

Pushmitha V. (234214X) — Dispatcher/Admin Module, Front End. Built the dispatcher and admin landing pages, versioned admin zone and rate commands, the merchant review, user and staff management and pricing editor screens, the `adminDashboard` resolver and the daily outcomes reports with reporting-period utilities; added capacity benchmarks.

Sinthuha N. (234197U) — Customer Module, Backend and Accounts Domain. Implemented the Delivery booking foundation, customer history and receipts, the PDF receipt adapter, authorised parcel and gRPC reads, the recipient authorisation read and secure public tracking; added cross-protocol contract regressions, the booking-to-delivery end-to-end test and post-rollback consistency checks.

Harithkeshan R. (234077D) — Dispatcher/Admin Module, Backend and Report Lead. Implemented staff read models and provisioning, assembled the first report draft, bootstrapped Pricing & Zones with versioned rates and expiring quotes, scaffolded the read-only GraphQL Query API and implemented the dispatcher queue with manual assignment and admin staff lifecycle APIs.

Praveen APE (234163M) — Merchant Module, Front End, Webhooks and Release QA. Built the merchant landing page, signed webhook attempts with response classification and the durable retry scheduler, the `merchantDashboard` resolver and the merchant order dashboard; automated merchant,

webhook and operator acceptance tests and coordinated release-candidate acceptance.

12 Appendix B – Running the System

The system runs from a clean clone with Docker (about 4 CPUs and 4 GB of Docker memory are recommended).

```
git clone https://github.com/Suthankan1/swiftdrop.git
cd swiftdrop
./platform/compose/start.sh          # all databases, Kafka, MinIO, Mailpit, Keycloak and services
python3 platform/identity/seed-demo-accounts.py --yes   # one account per role
./platform/compose/status.sh         # container health
./platform/compose/stop.sh           # stop, keeping data
```

After start-up the application is at http://localhost:3000, the gateway at http://localhost:8080, Keycloak at http://localhost:8090 and Mailpit at http://localhost:8025. Demo passwords are written to `platform/identity/.local/demo-accounts.json`. Google sign-in needs `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`, and the AI assistant needs `GEMINI_API_KEY`, in `platform/compose/.env`; these secrets are never committed.

13 Appendix C – Main API Endpoints

All paths are under `/api/v1` through the gateway. Booking and merchant order requests require an `Idempotency-Key` header.

Method and path	Purpose	Caller
`GET`, `PATCH /users/me`	Read or update own profile	Any signed-in user
`POST /merchants`; `GET /merchants/me`	Register merchant; own merchant	Customer, merchant
`POST /merchants/{id}/api-keys`; `…/{keyId}/revoke`	Create or revoke API keys	Approved merchant
`/merchants/webhooks`, `…/{id}/rotate-secret`, `…/{id}/deliveries`	Webhooks, secret rotation, delivery logs	Approved merchant
`GET /zones`; `POST /quotes`	Zones; price quote	Public, customer, merchant
`POST /parcels`; `GET /parcels[?scope=personal]`	Book; list own parcels	Customer
`POST /merchant/parcels`	Create an order	Merchant (session or API key)
`GET /parcels/{id}`, `…/history`, `…/receipt`, `…/events`	Details, timeline, PDF receipt, SSE stream	Owner, assigned courier, staff
`PATCH /parcels/{id}/status`; `POST /parcels/{id}/proof`	Status change; proof upload	Assigned courier

Method and path	Purpose	Caller
`POST /assignments`; `GET /assignments`; `GET /assignments/my`	Assign; active assignments; own assignments	Dispatcher, admin; courier
`POST /assignments/{id}/reassign`, `…/{id}/close`	Reassign; unassign	Dispatcher, admin
`GET /tracking/{trackingNumber}`, `…/events`	Public tracking and stream	Public
`/admin/**`	Users, staff, merchants, zones, rate cards	Admin
`POST /graphql`	Dashboards and statistics	Signed-in roles
`GET /assistant/status`; `POST /assistant/chat`, `/assistant/mcp`	AI assistant; MCP tools	Public and signed-in users

Table 7. Main API endpoints